The `Item` holds the entity disassembled state, as well as the entity version and a timestamp. The version and the timestamp are used for concurrency control as follows:

- An `Item` is readable only from a `Session` that has been started after the cache entry creation timestamp.
- An `Item` entry can be written only if the incoming version is greater than the current one held in the cache entry.

When an entity or a collection is either updated or deleted, Hibernate replaces the cached `Item` entry with a `Lock`, whose concurrency control mechanism works as follows:

- Since it overwrites an `Item` cache entry, the `Lock` object instructs a concurrent `Session` to read the entity or the collection from the database.
- If at least one `Session` has managed to lock this entry, any write operation is forbidden.
- A `Lock` entry is writable only if the incoming entity state has a version which is newer than the one contained in the `Lock` object, or if the current `Session` creation timestamp is greater than the `Lock` timeout threshold.

If the database transaction is rolled back, the current cache entry holds a `Lock` instance which cannot be undone to the previous `Item` state. For this reason, the `Lock` must time out to allow the cache entry to be replaced by an actual `Item` cache entry.

For Ehcache, the default `Lock` timeout is 120 seconds, and it can be customized via the `net.sf.ehcache.hibernate.cache_lock_timeout` configuration property.

The `READ_WRITE` concurrency strategy offers a write-through caching mechanism without requiring JTA transactions.

However, for heavy write contention scenarios, when there is a chance of rolling back transactions, the soft locking concurrency control can lead to having other concurrent transactions hitting the database for the whole duration of the lock timeout period. For this kind of situations, the `TRANSACTIONAL` concurrency strategy might be more suitable.

15.5.6.4 TRANSACTIONAL

While `READ_WRITE` is an asynchronous write-through cache concurrency strategy, `TRANSACTIONAL` uses a synchronous caching mechanism.

To enlist two data sources (the database and the second-level cache) in the same global transaction, a JTA transaction manager is needed. When using Java EE, the application server provides JTA transactions by default. For stand-alone enterprise applications, there are multiple transaction managers to choose from (e.g. Bitronix, Atomikos, Narayana).

For JTA transactions, Ehcache offers two failure recovery options: `xa_strict` and `xa`.

15.5.6.4.1 XA_Strict mode

In this mode, the second-level cache exposes a `XAResource` interface so that it can participate in the two-phase commit (2PC) protocol.

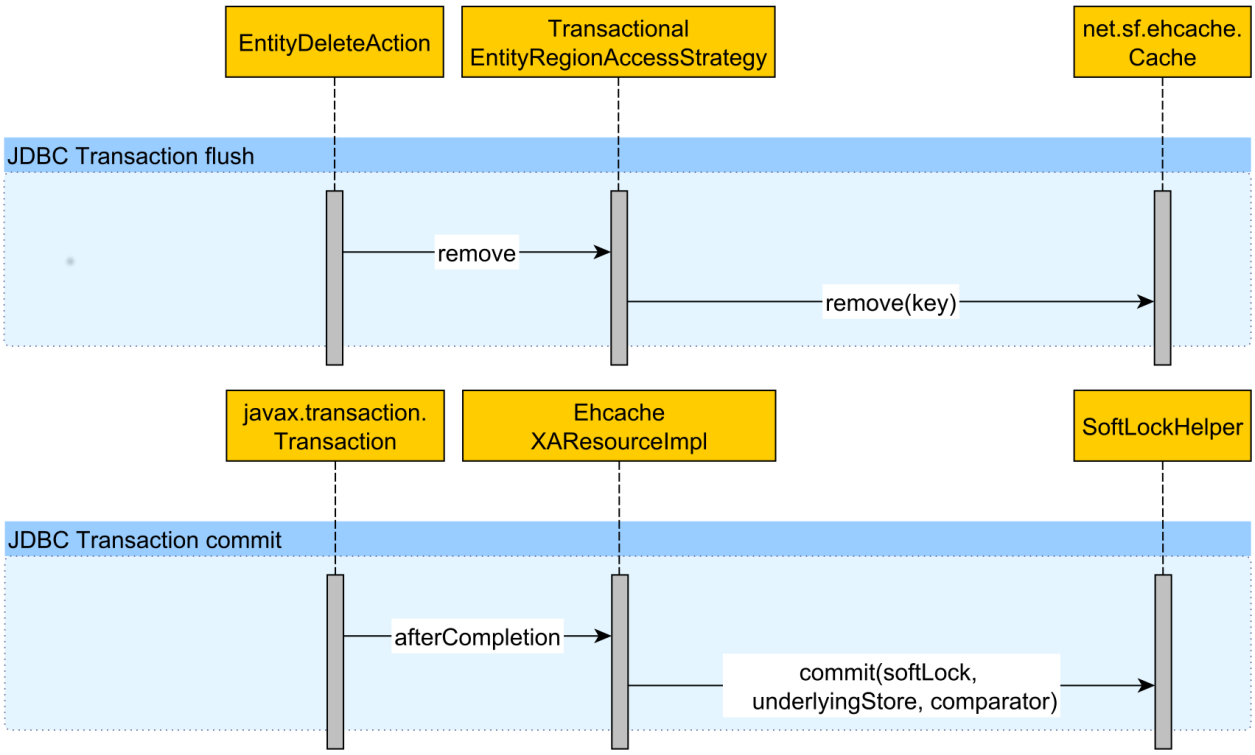

Figure 15.14: TRANSACTIONAL XA_Strict flow

The entity state is modified both in the database and in the cache, but these changes are isolated from other concurrent transactions, and they become visible once the current XA transaction gets committed.

The database and the cache remain consistent even in the case of an application crash.

15.5.6.4.2 XA mode

If only one `DataSource` participates in a global transaction, the transaction manager can apply the one-phase commit optimization. The second-level cache is managed through a `javax.transaction.Synchronization` transaction callback. The `Synchronization` does not actively participate in deciding the transaction outcome, therefore following the current database transaction outcome:

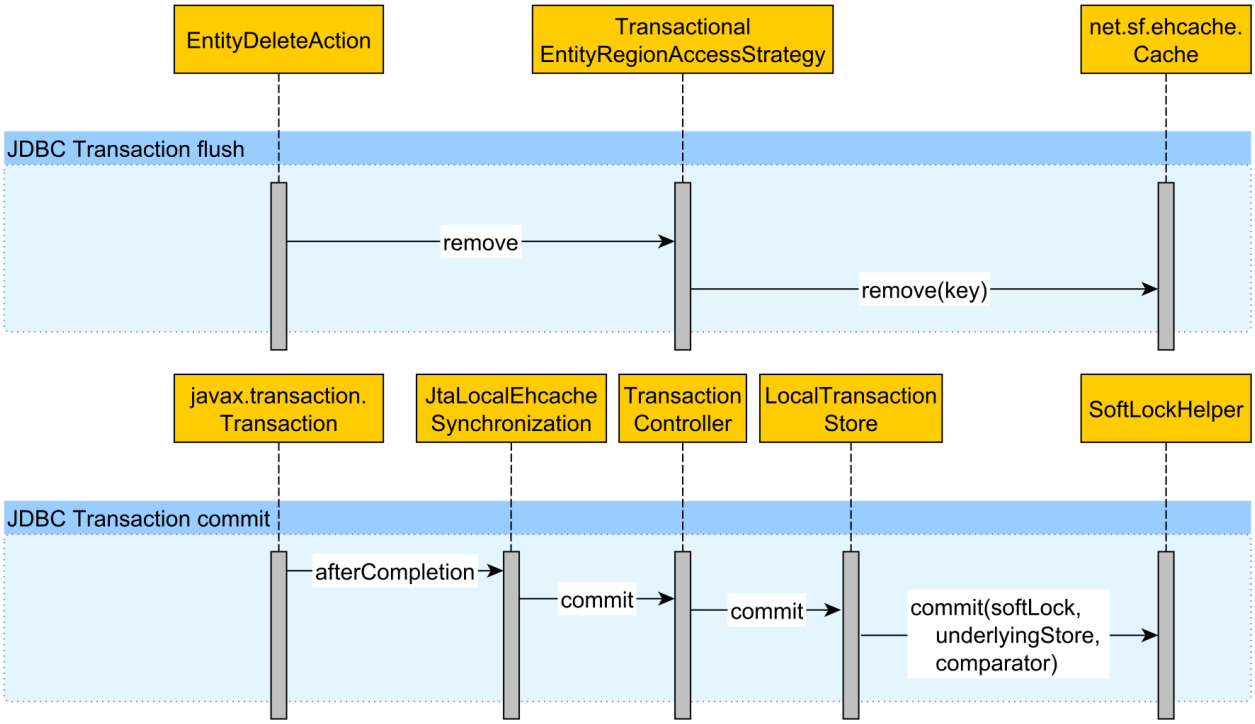

Figure 15.15: TRANSACTIONAL XA flow

This mode trades durability for a lower response time, and in the case of a server crash (happening in between the database transaction commit and the second-level cache transaction callback), the two data sources will drift apart. This issue can be mitigated if entities employ an optimistic concurrency control mechanism, so, even if the application reads stale data, it will not lose updates upon writing it back.

The following examples are going to reuse the same entities that were previously employed, and the only thing that differs is that the `Post` and `PostComment` entities, as well as the `comments` collections, are using the `@Cache(usage = CacheConcurrencyStrategy.TRANSACTIONAL)` annotation. The transaction boundaries are managed by Spring framework, and the actual JTA transaction logic is coordinated by Bitronix Transaction Manager.

15.5.6.4.3 Inserting TRANSACTIONAL cache entries

When running the same example used for inserting NONSTRICT_READ_WRITE cache entries, Hibernate generates the following output:

```
INSERT INTO post (title, version, id)
VALUES ('High-Performance Java Persistence', 0, 1)

INSERT INTO post_comment (post_id, review, id)
VALUES (1, 'JDBC part review', 1)
INSERT INTO post_comment (post_id, review, id)
VALUES (1, 'Hibernate part review', 2)

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=00],
Entries: {}

--Load Post entity and comments collection

SELECT pc.post_id AS post_id3_1_0_, pc.id AS id1_1_0_, pc.review AS review2_1_1_
FROM   post_comment pc
WHERE  pc.post_id = 1

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=CollectionCacheEntry[1,2]}
```

Just like READ_WRITE, the TRANSACTIONAL cache concurrency strategy is write-through for entities (unless using the identity generator in which case it is read-through), and read-though for collections.

15.5.6.4.4 Updating TRANSACTIONAL cache entries

Because the `TRANSACTIONAL` cache is synchronous, all changes are applied directly to cache, as illustrated by the following example:

```
doInJPA(entityManager -> {
    printCacheRegionStatistics(Post.class.getName());
    printCacheRegionStatistics(Post.class.getName() + ".comments");
    printCacheRegionStatistics(PostComment.class.getName());

    Post post = entityManager.find(Post.class, 1L);
    post.setTitle("High-Performance Hibernate");
    PostComment comment = post.getComments().remove(0);
    comment.setPost(null);

    entityManager.flush();

    printCacheRegionStatistics(Post.class.getName());
    printCacheRegionStatistics(Post.class.getName() + ".comments");
    printCacheRegionStatistics(PostComment.class.getName());

    LOGGER.debug("Commit after flush");
});
printCacheRegionStatistics(Post.class.getName());
printCacheRegionStatistics(Post.class.getName() + ".comments");
printCacheRegionStatistics(PostComment.class.getName());
```

For which Hibernate generates the following output:

```
Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=CollectionCacheEntry[1,2]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=2],
Entries: {1=CacheEntry(PostComment)[1,JDBC part review],
          2=CacheEntry(PostComment)[1,Hibernate part review]}
```

```
UPDATE post
SET title = 'High-Performance Hibernate', version = 1
WHERE id = 1 AND version = 0

DELETE FROM post_comment WHERE id = 1

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=2],
Entries: {1=CacheEntry(Post)[1,High-Performance Hibernate,1]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=1,putCount=1],
Entries: {}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=2],
Entries: {2=CacheEntry(PostComment)[1,Hibernate part review]}

--Commit after flush

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=2],
Entries: {1=CacheEntry(Post)[1,High-Performance Hibernate,1]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=1,putCount=1],
Entries: {}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=2],
Entries: {2=CacheEntry(PostComment)[1,Hibernate part review]}
```

Unlike the `READ_WRITE` cache concurrency strategy, `TRANSACTIONAL` does not use `Lock` cache entries, but instead it offers transaction isolation through the second-level cache provider internal locking mechanisms. After the `Post` entity and the `comments` collections are modified, Hibernate applies all the changes synchronously.

The `Post` entity modification is immediately visible in the cache, but only for the currently running transaction. Other transactions will not see any pending modifications until the current transaction is committed.

The `PostComment` entity that was deleted from the database is going to be removed from the entity cache region as well.

The `Post.comments` collection cache region is invalidated, and all its content is being removed.

From the current running transaction perspective, the `TRANSACTIONAL` cache concurrency strategy offers read-your-own-writes consistency guarantees. Once the transaction is com-

mitted, all pending database and cache changes are becoming visible to other concurrent transactions as well.

15.5.6.4.5 Deleting TRANSACTIONAL cache entries

When running the same example used for deleting `NONSTRICT_READ_WRITE` cache entries, Hibernate generates the following output:

```
Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=CollectionCacheEntry[1,2]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=2],
Entries: {1=CacheEntry(PostComment)[1,JDBC part review],
          2=CacheEntry(PostComment)[1,Hibernate part review]}

DELETE FROM post_comment WHERE id = 1
DELETE FROM post_comment WHERE id = 2
DELETE FROM post WHERE id = 1 AND version = 0

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=1],
Entries: {}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=1,putCount=1],
Entries: {}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=2,missCount=0,putCount=2],
Entries: {}
```

Unlike the `READ_WRITE` cache concurrency strategy which replaces the deleted `Item` cache entry with a `Lock` object, `TRANSACTIONAL` removes all the previously stored cache entries.

Choosing the right cache concurrency strategy

The concurrency strategy choice is based on the underlying data access patterns, as well as on the current application consistency requirements. By analyzing the second-level cache statistics, the application developer can tell how effective a cache concurrency strategy

renders. A high `hitCount` number indicates that the data access layer benefits from using the current cache concurrency strategy, while a high `missCount` value tells the opposite.

Although analyzing statistics is the best way to make sure that a strategy is a right choice, there are still some general guidelines that can be used to narrow the choice list.

For immutable data, the `READ_ONLY` strategy makes much sense because it even disallows updating cache entries.

If entities are changed infrequently and reading a stale entry is not really an issue, then the `NONSTRICT_READ_WRITE` concurrency might be a good candidate.

For strong consistency, the data access layer can either use `READ_WRITE` or `TRANSACTIONAL`. `READ_WRITE` is a good choice when the volume of write operations, as well as the chance of rolling back transaction, are rather low.

If the read and write ratio is balanced, `TRANSACTIONAL` might be a good alternative because updates are applied synchronously. If the roll back ratio is high (e.g. due to optimistic locking exceptions), the `TRANSACTIONAL` strategy is a much better choice because it allows rolling back cache entries, unlike `READ_WRITE` cache mode which maintains a `Lock` entry until it times out. Depending on the caching provider, even the `TRANSACTIONAL` cache concurrency strategy might offer different consistency modes (e.g. `xa`, `xa_strict`) so that the application developer can balance strong consistency with throughput. To overcome the overhead of the two-phase commit protocol, the Ehcache XA mode can leverage the one-phase commit optimization.

The concurrency strategy choice might be affected by the second-level cache topology as well. If the volume of data is high, a single node might not be sufficient, so data needs to be distributed across multiple nodes. A distributed cache increases cache availability because, if one node crashes, the cached data still lives on other machines. However, most distributed second-level cache providers do not support the `TRANSACTIONAL` cache concurrency strategy, leaving the application developer to choose either `NONSTRICT_READ_WRITE` or `READ_WRITE`.

15.5.7 Query cache strategy

The query cache does not take into consideration the cache concurrency strategy of the associated cached entities, so it has its own rules when it comes to ensuring data consistency. Just like the collection cache, the query cache uses a read-through approach, so queries are cached upon being executed for the first time.

`org.hibernate.cache.internal.StandardQueryCache` is the second-level cache region where query results are being stored.

To visualize how the read-through query cache works, consider the following query:

```
public List<PostComment> getLatestPostComments(EntityManager entityManager) {
    return entityManager.createQuery(
        "select pc " +
        "from PostComment pc " +
        "order by pc.post.id desc", PostComment.class)
    .setMaxResults(10)
    .setHint(QueryHints.HINT_CACHEABLE, true)
    .getResultList();
}
```

The `QueryHints.HINT_CACHEABLE` constant can be used to supply the JPA query hint that enables the second-level query cache.

If the current database contains the following entities:

```
Post post = new Post();
post.setId(1L);
post.setTitle("High-Performance Java Persistence");

PostComment comment = new PostComment();
comment.setId(1L);
comment.setReview("JDBC part review");
post.addComment(comment);

entityManager.persist(post);
```

When running the aforementioned query and printing the associated query cache region statistics:

```
doInJPA(entityManager -> {
    printCacheRegionStatistics(StandardQueryCache.class.getName());
    assertEquals(1, getLatestPostComments(entityManager).size());
    printCacheRegionStatistics(StandardQueryCache.class.getName());
});
```

Hibernate generates the following output:

```
Region: org.hibernate.cache.internal.StandardQueryCache,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=0],
Entries: {}

SELECT pc.id AS id1_1_, pc.post_id AS post_id3_1_, pc.review AS review2_1_
FROM   post_comment pc ORDER BY pc.post_id DESC LIMIT 10

Region: org.hibernate.cache.internal.StandardQueryCache,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {sql: ; named parameters: {}; max rows: 10; = [5992563495481345, 1]}
```

For brevity, the query cache entry was shortened. As expected, once the query is being executed, the matching entity identifiers are stored in the query cache entry.

15.5.7.1 Tablespace query cache invalidation

To understand query cache invalidation, considering the following exercise:

```
doInJPA(entityManager -> {
    assertEquals(1, getLatestPostComments(entityManager).size());
    printCacheRegionStatistics(StandardQueryCache.class.getName());

    LOGGER.info("Insert a new PostComment");
    PostComment newComment = new PostComment();
    newComment.setId(2L);
    newComment.setReview("JDBC part review");
    Post post = entityManager.find(Post.class, 1L);
    post.addComment(newComment);
    entityManager.flush();

    assertEquals(2, getLatestPostComments(entityManager).size());
    printCacheRegionStatistics(StandardQueryCache.class.getName());
});
LOGGER.info("After transaction commit");
printCacheRegionStatistics(StandardQueryCache.class.getName());

doInJPA(entityManager -> {
    LOGGER.info("Check query cache");
    assertEquals(2, getLatestPostComments(entityManager).size());
});
printCacheRegionStatistics(StandardQueryCache.class.getName());
```

Hibernate generates the following output:

```
Region: org.hibernate.cache.internal.StandardQueryCache,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {sql: ; named parameters: {}; max rows: 10;=[5992617470844929, 1]}

-- Insert a new PostComment

INSERT INTO post_comment (post_id, review, id)
VALUES (1, 'JDBC part review', 2)

UpdateTimestampsCache - Pre-invalidating space [post_comment],
timestamp: 5992617717362689
UpdateTimestampsCache - [post_comment] last update timestamp: 5992617717362689,
result set timestamp: 5992617470844929
StandardQueryCache - Cached query results were not up-to-date

SELECT pc.id AS id1_1_, pc.post_id AS post_id3_1_, pc.review AS review2_1_
FROM   post_comment pc ORDER BY pc.post_id DESC LIMIT 10

Region: org.hibernate.cache.internal.StandardQueryCache,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=2,putCount=2],
Entries: {sql: ; named parameters: {}; max rows: 10; =[5992617470844929, 2, 1]}

UpdateTimestampsCache - Invalidating space [post_comment],
timestamp: 5992617471619075

--After transaction commit

Region: org.hibernate.cache.internal.StandardQueryCache,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=2,putCount=2],
Entries: {sql: ; named parameters: {}; max rows: 10; =[5992617470844929, 2, 1]}

--Check query cache

StandardQueryCache - Checking query spaces are up-to-date: [post_comment]
UpdateTimestampsCache - [post_comment] last update timestamp: 5992617471619075,
result set timestamp: 5992617470844929
StandardQueryCache - Cached query results were not up-to-date

Region: org.hibernate.cache.internal.StandardQueryCache,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=3,putCount=3],
Entries: {sql: ; named parameters: {}; max rows: 10;=[5992617471627265, 2, 1]}
```

Hibernate second-level cache favors strong-consistency and the query cache is no different.

Whenever tablespaces are changing, the query cache invalidates all entries that are using the aforementioned tablespaces. The flow goes like this:

- Once the `PostComment` persist event is flushed, Hibernate pre-invalidates the `post_comment` tablespace timestamp (5992617717362689).
- When a tablespace is pre-invalidated, its timestamp is set to the cache region timeout timestamp value, which, by default, is set to 60 seconds.
- The query cache compares the cache entry timestamp with the tablespace pre-invalidation timeout timestamp value.
- Because the `post_comment` tablespace timestamp (5992617717362689) is greater than query result fetch timestamp (5992617470844929), the query cache ignores the cached entry value, and Hibernate executes the database query.
- The result set that is now fetched from the database goes to the cache without updating the result set timestamp (5992617470844929).
- When the current database transaction is committed, the `post_comment` tablespace is invalidated. Therefore, the tablespace timestamp is set to the transaction commit timestamp (5992617471619075).
- Even after the current database transaction is committed, the query cache timestamp is still seeing the old query result (5992617470844929).
- A new `Session` wants to execute the query, and because the query cache timestamp (5992617471619075) is still older than the `post_comment` tablespace timestamp, Hibernate executes the database query.
- Because this `Session` has not modified any tablespace, Hibernate updates the query cache with the current result set and the cache entry timestamp is set to the current `Session` timestamp (5992617471627265).

This flow guarantees strict consistency, and the query cache timestamp acts like a soft locking mechanism, preventing other concurrent transactions from reading stale entries.

15.5.7.2 Native SQL statement query cache invalidation

Hibernate can only parse JPQL and HQL statements, so it knows what tablespaces are required by a particular entity statement. For native statements, Hibernate cannot know if a tablespace is going to be affected directly or indirectly, and, by default, every native update statement is going to invalidate all query cache entries.

When executing the following example:

```
assertEquals(1, getLatestPostComments(entityManager).size());
printCacheRegionStatistics(StandardQueryCache.class.getName());

entityManager.createNativeQuery(
    "UPDATE post SET title = '\"'||title||'\"' ")
.executeUpdate();

assertEquals(1, getLatestPostComments(entityManager).size());
printCacheRegionStatistics(StandardQueryCache.class.getName());
```

Hibernate generates the following output:

```
Region: org.hibernate.cache.internal.StandardQueryCache,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {sql: ; named parameters: {}; max rows: 10;=[5992657080082432, 1]}

UpdateTimestampsCache - Pre-invalidating space [post_comment],
timestamp: 5992657328578560
UpdateTimestampsCache - Pre-invalidating space [post],
timestamp: 5992657328578560

UPDATE post SET title = '"'||title||'"'

StandardQueryCache - Checking query spaces are up-to-date: [post_comment]
UpdateTimestampsCache - [post_comment] last update timestamp: 5992657328578560,
result set timestamp: 5992657080082432
StandardQueryCache - Cached query results were not up-to-date

SELECT pc.id AS id1_1_, pc.post_id AS post_id3_1_, pc.review AS review2_1_
FROM   post_comment pc
ORDER BY pc.post_id DESC
LIMIT 10

Region: org.hibernate.cache.internal.StandardQueryCache,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=2,putCount=2],
Entries: {sql: ; named parameters: {}; max rows: 10;=[5992657080082432, 1]}

UpdateTimestampsCache - Invalidating space [post], timestamp: 5992668528799744
UpdateTimestampsCache - Invalidating space [post_comment],
timestamp: 5992668528799744
```

The flow goes like this:

- Initially, the query result is stored in the cache.
- Upon executing the native DML statement, Hibernate pre-invalidates all tablespaces (e.g. `post` and `post_comment`).
- When the data access layer executes the previously cached `PostComment` query, Hibernate checks the cache entry timestamp validity.
- Because the `post_comment` timestamp was set to the timeout value, Hibernate is prevented from using the cached result, so it executes the database query.
- When the transaction is committed, all tablespaces are invalidated, their associated timestamps being set to the current transaction commit timestamp.

To prevent Hibernate from invalidating all entries in the `StandardQueryCache` region, the native query must explicitly specify the tablespaces that are going to be affected:

```
entityManager.createNativeQuery(
    "UPDATE post SET title = '\"'||title||'\"' ")
.unwrap(SQLQuery.class).addSynchronizedEntityClass(Post.class)
.executeUpdate();
```

This time, Hibernate generates the following output:

```
Region: org.hibernate.cache.internal.StandardQueryCache,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {sql: ; named parameters: {}; max rows: 10;=[5992666396459009, 1]}

UpdateTimestampsCache - Pre-invalidating space [post],
timestamp: 5992666644185088

UPDATE post SET title = '"'||title||'"'

StandardQueryCache - Checking query spaces are up-to-date: [post_comment]
UpdateTimestampsCache - [post_comment] last update timestamp: 5992666396422146,
result set timestamp: 5992666396459009
StandardQueryCache - Returning cached query results

Region: org.hibernate.cache.internal.StandardQueryCache,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=1,putCount=1],
Entries: {sql:; named parameters: {}; max rows: 10;f2=[5992666396459009, 1]}

UpdateTimestampsCache - Invalidating space [post], timestamp: 5992666398470144
```

Because this time only the `post` tablespace is invalidated, and since the entity query uses the `post_comment` table, the previously cached query result can be reused to satisfy the current entity query fetching requirements.

Query cache applicability

As explained in the Fetching chapter, DTO projections are suitable for executing read-only queries. For this purpose, the query cache is not a general purpose solution since it can only select entities.

However, fetching entities is appropriate for read-write transactions, and any entity modification can trigger a ripple effect in the `StandardQueryCache` second-level cache region. For this purpose, the query cache works better for immutable entities, or for entities that rarely change.

16. Concurrency Control

As explained in the JDBC Transactions chapter, every SQL statement executes within the scope of a database transaction. To prevent conflicts, database engines employ row-level locks. Database physical locks can either be acquired implicitly or explicitly. Whenever a row is changed, the relational database acquires an implicit exclusive lock on the aforementioned record to prevent write-write conflicts.

Locks can also be acquired explicitly, in which case the concurrency control mechanism is called pessimistic locking. Exclusive locks can be acquired explicitly on most database systems, whereas shared locks are not universally supported.

Pessimistic locking deals with concurrency conflicts through prevention, which can impact application performance and scalability. For this reason, to increase transaction throughput while still ensuring strong consistency, many data access frameworks provide optimistic locking support as well.

16.1 Hibernate optimistic locking

Even if pessimistic locking has only been added in JPA 2.0, optimistic locking has been supported since version 1.0. Just like pessimistic locking, the optimistic concurrency control mechanism can be used implicitly or explicitly.

16.1.1 The implicit optimistic locking mechanism

To enable the implicit optimistic locking mechanism, the entity must provide a `@Version` attribute:

```
@Entity @Table(name = "post")
public class Post {

    @Id
    private Long id;

    private String title;

    @Version
    private int version;

    //Getters and setters omitted for brevity
}
```

Logical vs. Physical clocks

Using timestamps to order events is rarely a good idea. System time is not always monotonically incremented, and it can even go backward due to network time synchronization (NTP protocol).

More, time accuracy across different database systems varies from nanoseconds (Oracle[a]) to 100 nanoseconds (SQL Server[b]), to microseconds (PostgreSQL[c] and MySQL 5.6.4[d] and even seconds (previous versions of MySQL). In distributed systems, logical clocks (e.g. vector clocks or Lamport timestamps) are always preferred to physical timestamps (wall clocks) when it comes to ordering events.

For this reason, employing a numerical version is more appropriate than using a timestamp.

[a] http://docs.oracle.com/database/121/LNPCB/pco04dat.htm#LNPCB269
[b] https://msdn.microsoft.com/en-us/library/bb677335.aspx
[c] http://www.postgresql.org/docs/current/static/datatype-datetime.html
[d] http://dev.mysql.com/doc/refman/5.6/en/fractional-seconds.html

To visualize the optimistic concurrency control, when executing the following test case:

```
doInJPA(entityManager -> {
    Post post = new Post();
    post.setId(1L);
    post.setTitle("High-Performance Java Persistence");
    entityManager.persist(post);

    entityManager.flush();
    post.setTitle("High-Performance Hibernate");
});
```

Hibernate generates the following output:

```
INSERT INTO post (title, version, id)
VALUES ('High-Performance Java Persistence', 0, 1)

UPDATE post SET title = 'High-Performance Hibernate', version = 1
WHERE id = 1 AND version = 0
```

Whenever an update occurs, Hibernate is going to filter the database record according to the expected entity version. If the version has changed, the update count is going to be 0, and a `OptimisticLockException` is going to be thrown.

To visualize the conflict detection mechanism, consider the following exercise:

```
doInJPA(entityManager -> {
    Post post = entityManager.find(Post.class, 1L);
    executeSync(() -> {
        doInJPA(_entityManager -> {
            Post _post = _entityManager.find(Post.class, 1L);
            _post.setTitle("High-Performance JDBC");
        });
    });
    post.setTitle("High-Performance Hibernate");
});
```

When executing the aforementioned test case, Hibernate generates the following output:

```
-- Alice selects the Post entity
SELECT p.id AS id1_0_0_, p.title AS title2_0_0_, p.version AS version3_0_0_
FROM   post p
WHERE  p.id = 1

-- Bob also selects the same Post entity
SELECT p.id AS id1_0_0_, p.title AS title2_0_0_, p.version AS version3_0_0_
FROM   post p
WHERE  p.id = 1

-- Bob updates the Post entity
UPDATE post SET title = 'High-Performance JDBC', version = 1
WHERE id = 1 AND version = 0

-- Alice also wants to update the Post entity
UPDATE post SET title = 'High-Performance Hibernate', version = 1
WHERE id = 1 AND version = 0

--Exception thrown
javax.persistence.RollbackException: Error while committing the transaction
Caused by: javax.persistence.OptimisticLockException:
Caused by: org.hibernate.StaleStateException:
Batch update returned unexpected row count from update [0];
actual row count: 0; expected: 1
```

Because this example uses the Java Persistence API, the Hibernate internal `StaleStateException` is wrapped in the `OptimisticLockException` defined by the JPA specification.

The flow of operations can be summarized as follows:

- Alice fetches a `Post` entity and then her thread is suspended.
- Bob thread is resumed, he fetches the same `Post` entity and changes the `title` to `High-Performance JDBC`. The entity `version` is set to 1.
- When Alice's thread is resumed, she tries to update the `Post` entity `title` to `High-Performance Hibernate`.
- An `OptimisticLockException` is thrown because the second update statement is expecting to filter the entity version with a value of 0, while the `version` column value is now 1.

16.1.1.1 Resolving optimistic locking conflicts

While pessimistic locking prevents conflict occurrences, optimistic locking mechanisms, just like MVCC, use conflict detection instead. So anomalies are detected and prevented from being materialized by aborting the currently running transactions, and Hibernate optimistic locking can prevent the *lost update* anomaly.

As explained in the application-level transactions section, when using a multi-request workflow, the database isolation level can no longer prevent lost updates. On the other hand, the optimistic locking mechanism can prevent losing updates as long as the entity state is preserved from one request to the other.

Optimistic locking discards all incoming changes that are relative to a stale entity version. However, everything has its price and optimistic locking is no different.

If two concurrent transactions are updating distinct entity attribute subsets, then there should be no risk of losing any update. However, the optimistic concurrency control mechanism takes an all-or-nothing approach even for non-overlapping changes. For this reason, two concurrent updates, both starting from the same entity version, are always going to collide. It is only the first update that is going to succeed, the second one failing with an optimistic locking exception.

This strict policy acts as if all changes are going to overlap, and, for highly concurrent write scenarios, the single version strategy can lead to a large number of transactions being rolled back.

To visualize the non-overlapping conflict, consider the following `Post` entity class:

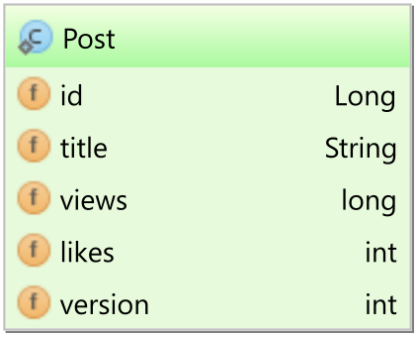

Figure 16.1: Post entity with a single global version

In the following example, Alice modifies the `Post` entity `title` attribute, Bob increments the `likes` counter, and Carol sets the `views` attribute to a value that was aggregated from an external batch processor.

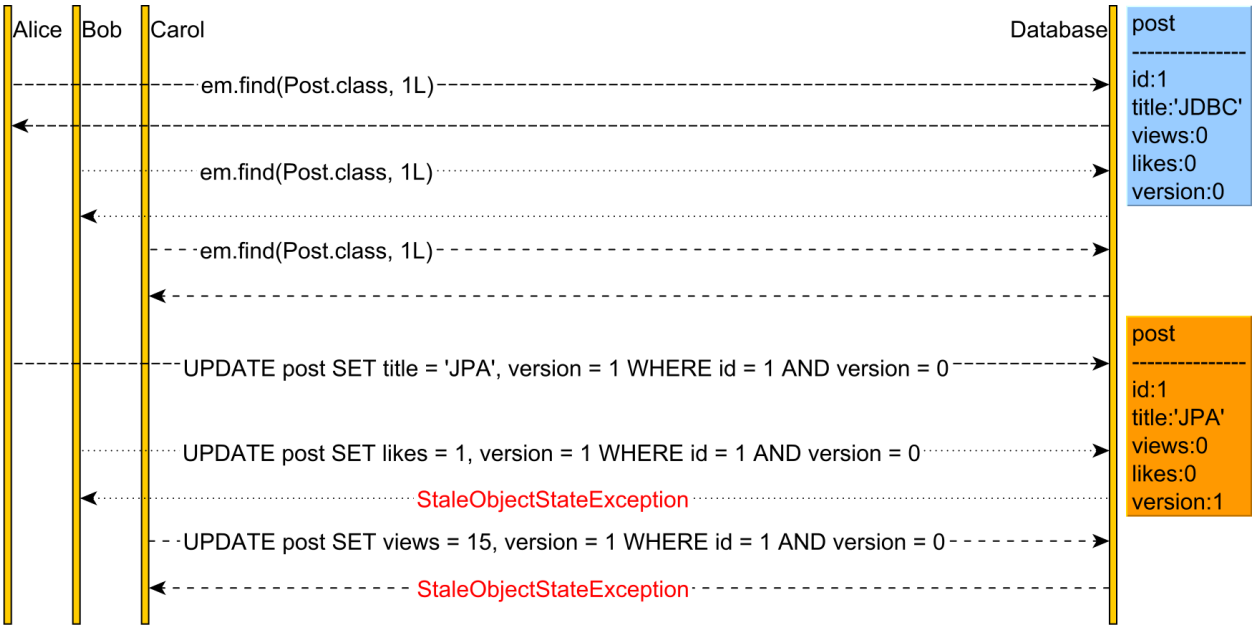

Figure 16.2: Optimistic locking non-overlapping conflict

The flow of operations can be explained as follows:

- All three users are loading the same `Post` entity version.
- Alice modifies the `title`. Therefore, the `Post` entity `version` is incremented.
- Bob tries to increment the `likes` counter but rolls back because it expects the `version` to be 0, but now it has a value of 1.
- Carol's transaction is also aborted because of the entity version mismatch.

The optimistic locking mechanism allows only monotonically increasing version updates. If changes were dependent one to another, then getting an `OptimisticLockException` is less of an issue than losing an update. However, if from a business logic perspective, the changing attributes are not overlapping, having a single global version is no longer sufficient.

For this reason, the single global version must be split into multiple subversions, and this can be done in two ways:

- Instead of having a single optimistic locking counter, there can be a distinct version for each individual attribute set.
- Each changing attribute can be compared against its previously known value, so lost updates are relative only to the attribute in question.